📌 Extract Numerical & Categorical Values from a Mixed Column

Goal: Separate numeric and non-numeric values from the same column.

Extract Numerical Values

```
df['number_numerical'] = pd.to_numeric(
    df['number'],
    errors='coerce',
    downcast='integer'
)
```

Parameters / Keywords

- `arg` : `df['number']` → Input column.
- `errors='coerce'` → Invalid values become `NaN`.

Other options:

- `errors='raise'` *(default)* → Raises an error.
- `errors='ignore'` → Returns original values (deprecated).
- `downcast='integer'` → Use the smallest integer type.

Other options:

- `'signed'`
- `'unsigned'`
- `'float'`

Extract Categorical Values

```
df['number_categorical'] = np.where(
    df['number_numerical'].isnull(),
    df['number'],
    np.nan
)
```

Keywords

- `isnull()` → Detect missing values (`NaN`).

Similar methods:

- `isna()`
- `notnull()`

- `notna()`

- `np.where(condition, x, y)` → Vectorized if-else operation.

Logic

```
Valid Number   → number_numerical
Invalid Value  → NaN → number_categorical
```

Quick Revision

- `pd.to_numeric()` → String ➜ Number
- `coerce` → Invalid ➜ `NaN`
- `raise` → Show error
- `ignore` → Keep original values
- `isnull()` / `isna()` → Find missing values
- `np.where()` → Apply if-else on arrays
- `downcast` → Reduce memory usage

Common Use Cases

- Data cleaning
- Feature engineering
- Handling mixed-type columns
- Preprocessing before machine learning